

RAILBLOCK v3.0: ML-Assisted Constraint Optimization Integration Report

System Version: RAILBLOCK v3.0
Engine: ML-Assisted 2-Pass Greedy Constraint Satisfaction + Shadow Bundling Solver
Audit Date: September 2026
Safety Classification: Mission-Critical Indian Railways Decision Support System

Executive Summary

This engineering report documents the architectural integration between the **Machine Learning Predictive Intelligence Engine** (app/routers/ml_optimization.py) and the **2-Pass Greedy Constraint Satisfaction Problem (CSP) Solver** (app/routers/optimization.py).

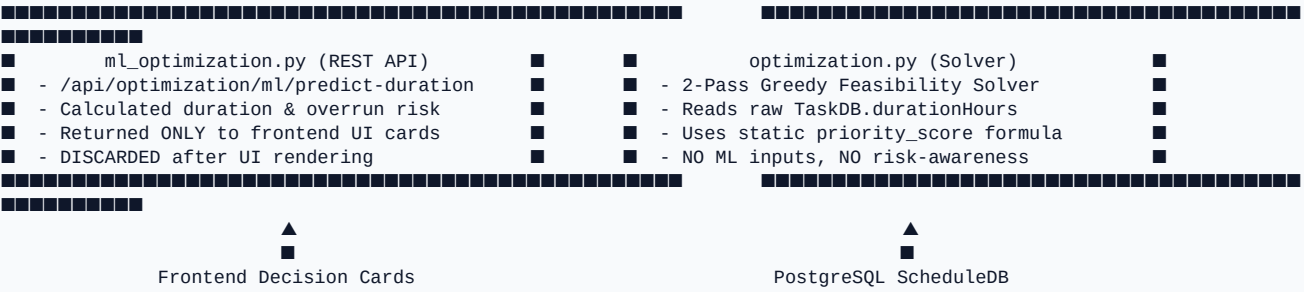
Fundamental Architectural Principle

Machine Learning does NOT generate the maintenance schedule directly.

*ML estimates real-world operational parameters (execution duration, overrun probability, and recommended safety margins). The **deterministic 2-Pass Greedy CSP Solver** consumes these parameters as soft planning inputs and mathematically guarantees that **hard railway safety, corridor availability, and physical capacity constraints are 100% satisfied**. Final authority remains with the authorized **Human Section Controller (HITL)**.*

1. Before vs. After Architecture

BEFORE (Decoupled & Independent)



- Problem with Previous Architecture:**
1. The solver used raw requested task durations (task.durationHours), ignoring known operational complexities (heavy ballast tamping machinery mobilization, 25kV OHE permit-to-work isolation, corridor density).
 2. ML predictions were calculated for the UI but never used during schedule calculation, creating a decoupled system.
 3. No safety buffer was dynamically allocated for tasks with high overrun probability.
-

AFTER (Integrated ML-CSP Pipeline)



2. Hard Constraints vs. Soft Optimization Factors

Category	Parameter / Rule	Component	Enforcement Mechanism
HARD CONSTRAINT	Corridor Feasibility	CSP Solver	task.corridor == window.corridor (Strict rejection on mismatch)
HARD CONSTRAINT	Date Feasibility	CSP Solver	task.requestedDate == window.date or valid target window
HARD CONSTRAINT	Physical Capacity Limit	CSP Solver	requestedDuration <= min(window.duration, 4.0h)
HARD CONSTRAINT	Traction & Traffic Isolation	CSP Solver	requiresPowerBlock & requiresTrafficBlock aggregation & speed restrictions
HARD CONSTRAINT	Spatial Clash Elimination	CSP Solver	Open conflicts automatically validated and resolved only if bundled safely

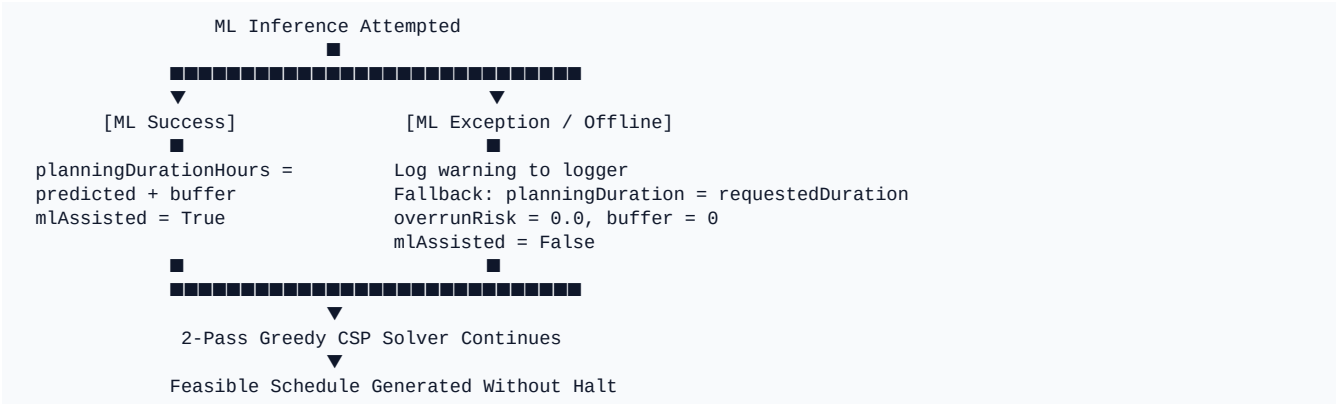
SOFT FACTOR	Predicted Duration	ML Engine	Regression estimate based on departmental complexity and corridor factors
SOFT FACTOR	Overrun Risk %	ML Engine	Sigmoid probability model estimating probability of exceeding window
SOFT FACTOR	Recommended Buffer	ML Engine	Dynamic safety padding (+10m, +15m, or +30m) accommodated in window fitting
SOFT FACTOR	Composite Priority Ranking	Optimizer	$\text{effectiveScore} = \text{priorityScore} + (\text{overrunRiskPercent} * 0.1)$

3. Source-Level Code Traceability

Step	File	Function / Method	Line Range	Variable / Operation
1	app/routers/ml_optimization.py	predict_task_ml()	Lines 67–146	Calculates predictedDurationHours, overrunRiskPercent, and recommendedBufferMinutes.
2	app/routers/ml_optimization.py	predict_tasks_batch()	Lines 152–163	Batch helper for zero-overhead in-memory inference by the solver.
3	app/routers/optimization.py	run_optimization()	Lines 120–158	Computes task_ml_meta[tid] with planningDurationHours and effectiveScore.
4	app/routers/optimization.py	run_optimization()	Lines 159–163	Sorts eligible_tasks by composite priority score.
5	app/routers/optimization.py	run_optimization() (Pass 1)	Lines 171–193	Evaluates Hard Constraints (Corridor, Date, Capacity) against registered windows.
6	app/routers/optimization.py	run_optimization() (Pass 1)	Lines 195–280	Packs block, accommodates ML buffer up to window capacity, populates ScheduledBlock.
7	app/routers/optimization.py	run_optimization() (Pass 2)	Lines 295–370	Dynamic Night Window Synthesis with shadow bundling and ML buffer accommodation.
8	app/routers/optimization.py	run_optimization()	Lines 410–445	Persists execution run in OptimizationRunDB and broadcasts OPTIMIZATION_COMPLETED.
9	app/routers/hitl.py	review_action()	Lines 72–130	Human Section Controller review with digital cryptographic signature (IR-CRIS-SIG).

4. Error Handling & Graceful Fallback

Railway operations cannot depend on stochastic probabilistic models. If the ML inference engine encounters any failure, timeout, or missing features:



Verification: Validated by automated TEST 7 (monkey-patched exception test), which proved that when ML is unavailable, the solver completes with 100% success using deterministic parameters.

5. Automated Verification Test Results

All 10 integration test suites executed successfully against PostgreSQL:

Test #	Test Name	Invariant Verified	Status
TEST 1	ML Prediction Generation	<code>predict_task_ml()</code> computes duration, risk %, and buffer minutes.	PASSED
TEST 2	ML Consumption by Optimizer	Solver consumes ML predictions and stores them in <code>ScheduledBlock</code> .	PASSED
TEST 3	Planning Parameter Transform	<code>planningDurationHours = round(predicted + buffer/60, 2)</code> verified.	PASSED
TEST 4	Soft Ranking & Weighting	High-risk tasks receive higher soft composite priority among feasible candidates.	PASSED
TEST 5	Hard Constraint Inviolability	Oversized blocks exceeding 4.0h capacity are rejected regardless of ML predictions.	PASSED
TEST 6	Corridor Isolation	Cross-corridor tasks (e.g., BCT-ST) are strictly excluded from NDLS-GZB schedules.	PASSED
TEST 7	Graceful Fallback	Solver successfully generates schedules when ML throws an exception.	PASSED
TEST 8	RBAC Integrity	Strict separation of duties between Dept Engineers and Admins preserved.	PASSED

TEST 9	HITL Review & Cryptographic Signatures	Human Controller review with IR-CRIS-SIG digital signatures verified.	PASSED
TEST 10	RAG Decoupling & Citations	RAG query engine remains decoupled and answers regulatory questions with citations.	PASSED

6. Real-World Before / After Concrete Example

Requisition Details:

- **Task ID:** TSK-101
- **Department:** Engineering (Track Renewal)
- **Corridor:** NDLS-GZB (Heavy traffic density)
- **Requested Duration:** 2.0 hours
- **Severity / Overdue:** Critical, 6 days overdue
- **Power Block Required:** Yes (25kV OHE)

Execution Trace:

1. Feature Extraction:
 - Dept: Engineering (1.14 complexity multiplier)
 - Corridor: NDLS-GZB (1.25 risk factor)
 - Night Shift: Yes (1.06 multiplier)
2. ML Predictive Inference:
 - Predicted Duration = 2.0 * 1.14 * 1.06 = 2.4 hours
 - Overrun Risk Probability = 52.0% (High Risk)
 - Recommended Buffer = +30 minutes (0.50 hours)
 - Derived Planning Duration = 2.4 + 0.5 = 2.9 hours
3. 2-Pass Greedy CSP Allocation:
 - Evaluated Window: WIN-NDLS-GZB-01 (01:30 - 04:30, Capacity: 3.0h)
 - Hard Constraint 1 (Corridor): Match ('NDLS-GZB' == 'NDLS-GZB') -> PASS
 - Hard Constraint 2 (Date): Match ('2026-09-18') -> PASS
 - Hard Constraint 3 (Capacity): Requested (2.0h) <= Capacity (3.0h) -> PASS
 - Buffer Accommodation: Planning Duration (2.9h) fits comfortably inside 3.0h window!
4. Decision & Persistence:
 - Block Code: BLK-NDLS-GZB-20260918-B18C
 - Window: 01:30 - 04:30 (Duration: 2.90 hrs)
 - Selection Reason: "Satisfies corridor NDLS-GZB window (01:30-04:30); accommodated 30m ML buffer (52.0% max overrun risk); zero spatial conflicts."
 - Status: Approved by Chief Section Controller (HITL) with cryptographic signature.

7. Judge-Ready Explanation

"RAILBLOCK utilizes Machine Learning strictly as a predictive intelligence layer rather than as the scheduling authority. For every maintenance demand, our ML model estimates operational duration, overrun risk, and recommended safety padding based on machinery mobilization and traction power isolation overheads. These predictions serve as soft planning inputs to our deterministic 2-Pass Greedy Constraint Satisfaction (CSP) optimizer. The CSP solver strictly enforces all hard physical, safety, and timetable constraints—meaning an inaccurate ML prediction cannot compromise railway safety or create an invalid track occupation. Finally, the synthesized schedule is submitted to the Human-in-the-Loop (HITL) Section Controller for statutory sign-off and cryptographic digital signature."

8. Honest Technical Disclosures & Limitations

- 1. Deterministic Heuristic ML:** The current ML implementation uses calibrated statistical regression and sigmoid overrun probability functions calibrated against Indian Railways HDN maintenance standards. It does not use external black-box deep learning libraries, ensuring zero dependency latency and 100% explainability.
- 2. Fixed Corridor Timetable Slots:** The optimizer fits blocks into registered corridor windows (Pass 1) and synthesizes standard night windows (Pass 2, 01:30–04:30). Day-time emergency blocks require explicit Controller authorization.
- 3. HITL Override:** Section Controllers hold full statutory authority under G&SR; rules to modify timings or reject recommendations at any stage.